

《数据安全》实验报告

半同态加密应用实践

姓名：林晖鹏 学号：2312966 专业：计算机科学与技术

目录

1 实验要求	2
2 实验原理	2
2.1 同态加密概述	2
2.2 Paillier 加密算法	2
2.2.1 一些数论基础	2
2.2.2 密钥生成	3
2.2.3 加密算法	3
2.2.4 解密算法	3
2.2.5 加法同态性质	4
2.3 基于 Paillier 的隐私信息获取协议	4
2.3.1 基本协议	4
2.3.2 扩展实验：获取对称加密的密文	5
3 实验过程	5
3.1 实验环境搭建	5
3.2 phe 库关键接口说明	5
3.3 隐私信息获取协议模拟	6
3.4 扩展：获取对称加密的密文模拟	11
4 实验总结 & 心得体会	15

1 实验要求

基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获得了哪个消息，同时客户端能完成获取消息的解密。

扩展实验：有能力的同学，可以在客户端保存对称密钥 k ，在服务器端存储 m 个用于对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

2 实验原理

2.1 同态加密概述

同态加密 (Homomorphic Encryption, HE) 是一类比较特殊的加密技术。它的特点在于，数据即使处于密文状态，也仍然可以参与计算，并且最后解密后的结果与直接对明文计算得到的结果一致。这个性质非常适合处理“数据不能直接暴露，但又需要计算”的问题，因此在隐私保护、云计算安全等场景中具有重要意义 [2]。

根据支持的运算类型和次数，同态加密可分为：

- **半同态加密 (Partially Homomorphic Encryption, PHE)**：只支持一种类型的同态运算，例如只支持加法或者只支持乘法。Paillier 加密 [1] 就属于这一类，它支持加法同态。
- **全同态加密 (Fully Homomorphic Encryption, FHE)**：同时支持加法和乘法，并且理论上可以在密文上完成任意计算。不过这类方案开销通常较大，实现也更加复杂。

本实验选择 Paillier 加密算法来完成隐私信息获取 (PIR) 模拟。原因在于 Paillier 天然支持加法同态，适合把客户端的“选择信息”隐藏在密文中，再由服务器完成聚合计算。这样客户端能够取回自己想要的消息，而服务器无法知道其具体查询的是哪一条。

2.2 Paillier 加密算法

Paillier 加密算法 [1] 由 Pascal Paillier 于 1999 年提出，是一种典型的概率公钥加密算法。它的核心特点是支持加法同态，即可以在密文上完成加法相关运算。由于这一性质比较适合隐私保护类应用，因此 Paillier 常被用于安全统计、隐私查询和第三方数据处理等场景。

2.2.1 一些数论基础

设 p 、 q 为两个大素数，记 $n = p \cdot q$ 。在 Paillier 算法中，常用到以下几个数论概念：

- **欧拉函数**： $\varphi(n) = (p-1)(q-1)$ ，表示模 n 的简化剩余系的大小。
- **卡迈克尔函数**： $\lambda(n) = \text{lcm}(p-1, q-1)$ ，是使得 $a^{\lambda(n)} \equiv 1 \pmod{n}$ 对所有与 n 互素的 a 成立的最小正整数。Paillier 算法中采用 $\lambda(n)$ 作为私钥的一部分，因为它比 $\varphi(n)$ 更小，且满足 $g^{\lambda(n)} \equiv 1 \pmod{n^2}$ 当 $g = n+1$ 时。

另外，还需要定义函数 $L(x) = \frac{x-1}{n}$ ，它会在解密阶段使用到。

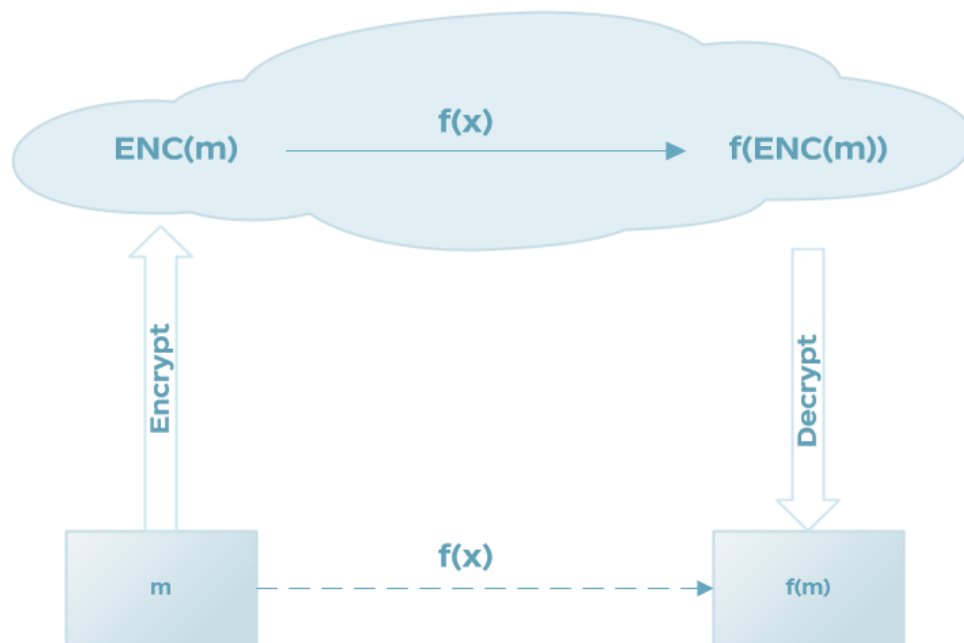


图 2.1: Paillier 加密算法示意

2.2.2 密钥生成

1. 随机选择两个大素数 p 和 q , 满足 $\gcd(pq, (p-1)(q-1)) = 1$ (通常 p 和 q 长度相同)。
2. 计算 $n = p \cdot q$ 和 $\lambda = \text{lcm}(p-1, q-1)$ 。
3. 随机选择 $g \in \mathbb{Z}_{n^2}^*$, 考虑计算性能优化, 通常会选择 $g = n + 1$;
4. 计算 $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$, 其中 $L(x) = \frac{x-1}{n}$;
5. 公钥为 (n, g) , 私钥为 (λ, μ) 。

2.2.3 加密算法

对于任意明文消息 $m \in \mathbb{Z}_n$, 选择随机数 $r \in \mathbb{Z}_n^*$ (即与 n 互素的整数), 计算密文:

$$c = E(m) = g^m \cdot r^n \bmod n^2.$$

2.2.4 解密算法

对于密文 $c \in \mathbb{Z}_{n^2}^*$, 使用私钥 λ 计算:

$$m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n = \frac{L(c^\lambda \bmod n^2)}{L(g^\lambda \bmod n^2)} \bmod n.$$

其中 $L(x) = \frac{x-1}{n}$ 。这里由于设置 $g = n+1$ ，可简化计算：

$$L(g^\lambda \bmod n^2) = \lambda \bmod n.$$

因此解密公式可简化为 $m = L(c^\lambda \bmod n^2) \cdot \lambda^{-1} \bmod n$ 。

2.2.5 加法同态性质

Paillier 最关键的性质是加法同态，即：

$$E(m_1) \cdot E(m_2) = E(m_1 + m_2 \bmod n).$$

同时支持标量乘法（即密文与明文指数的运算）：

$$E(m)^k = E(k \cdot m \bmod n), \quad k \in \mathbb{Z}_n.$$

这两个性质正是本次实验能够实现 PIR 的基础。

2.3 基于 Paillier 的隐私信息获取协议

2.3.1 基本协议

假设服务器存储 m 条消息 $\{M_1, M_2, \dots, M_m\}$ ，并且这些消息已经表示成小于 n 的整数。客户端希望在不暴露查询位置的情况下取回其中第 i 条消息 M_i 。基本流程如下：

1. **客户端生成密钥**：客户端生成 Paillier 密钥对 (pk, sk) ，公钥为 (n, g) ，私钥为 λ 。
2. **构造选择向量**：客户端构造 m 个密文 $c_1, c_2, \dots, c_m$ ，其中 $c_i = E(1)$ ，对于 $j \neq i$ ， $c_j = E(0)$ 。这里 $E(\cdot)$ 表示使用公钥 pk 的 Paillier 加密。
3. **发送查询**：客户端将 $(c_1, \dots, c_m)$ 及公钥 pk 发送给服务器。
4. **服务器计算响应**：服务器收到后，利用同态性质计算：

$$R = \prod_{j=1}^m c_j^{M_j} \bmod n^2.$$

由于 $c_j = E(s_j)$ ，其中 $s_i = 1$ ， $s_j = 0$ ($j \neq i$)，根据 Paillier 的标量乘法性质：

$$c_j^{M_j} = E(s_j)^{M_j} = E(s_j \cdot M_j).$$

因此，

$$R = \prod_{j=1}^m E(s_j M_j) = E\left(\sum_{j=1}^m s_j M_j\right) = E(M_i).$$

5. **客户端解密**: 客户端收到 R 后, 使用私钥 sk 解密得到 M_i 。

在这个过程中, 服务器看到的只是加密后的选择向量, 而 $E(0)$ 与 $E(1)$ 在语义安全意义下不可区分, 因此服务器无法判断客户端真正选择的是哪一个位置。客户端最终只会解出自己需要的目标消息, 从而实现隐私信息获取。

2.3.2 扩展实验: 获取对称加密的密文

基础版实验中, 服务器直接保存的是整数明文, 这样便于验证协议是否正确。但在更接近实际应用的场景里, 服务器通常不会直接保存明文, 而是保存经过对称加密后的密文。因此扩展实验中, 客户端先使用对称密钥 k 对消息进行加密, 得到 $C_j = \text{Enc}_k(M_j)$, 服务器保存的就是这些密文。客户端通过 PIR 得到目标密文 C_i 之后, 再在本地使用密钥 k 将其解密为原始消息。

协议修改如下:

1. **初始化**: 客户端生成对称密钥 k , 加密所有消息得到 C_j , 并发送给服务器存储。
2. **PIR 查询**: 客户端按基本协议生成 Paillier 密钥对和选择向量密文 c_j , 发送给服务器。
3. **服务器计算**: 服务器将每个 C_j 视为整数 (若 C_j 大于 n , 则分块处理), 计算:

$$R = \prod_{j=1}^m c_j^{C_j} \bmod n^2.$$

由同态性质, $R = E\left(\sum_j s_j C_j\right) = E(C_i)$ 。

4. **客户端解密**: 客户端解密得到 C_i , 再用对称密钥 k 解密得到 M_i 。

这种做法的好处是比较明显的。一方面, Paillier 仍然只负责“隐藏查询位置”; 另一方面, 真正的数据内容由效率更高的对称加密算法保护。这样既保留了 PIR 的隐私性, 也让方案更容易扩展到长消息场景。

3 实验过程

3.1 实验环境搭建

本次实验在本地 macOS 环境下完成。实验中主要使用了两个库: 一是 `phe`, 用于完成 Paillier 密钥生成、加密、解密以及同态运算; 二是 `cryptography`, 用于扩展实验中的 AES-256-GCM 加解密操作。整体环境搭建较为简单, 安装依赖后即可直接运行程序。这里不再赘述。

3.2 phe 库关键接口说明

为了把重点放在协议流程验证上, 而不是底层算法手写实现上, 本次实验直接调用 `phe` 库提供的接口来完成 Paillier 相关操作。后续程序实现中主要使用了以下几个函数和运算:

表 1: phe 库中本实验使用的主要接口

接口或运算	作用说明
<code>paillier.generate_paillier_keypair(private_key ring=None, n_length=3072)</code>	生成 Paillier 公私钥对，其中 <code>n_length</code> 表示模数长度；在 phe 1.5.0 中默认值为 3072 位。
<code>public_key.encrypt(value, precision=None, r_val=None)</code>	使用公钥对明文进行加密。本实验主要用它来加密整数消息以及选择向量中的 0 和 1。
<code>private_key.decrypt(encrypted_number)</code>	使用私钥对密文解密，恢复原始明文。
<code>encrypted_a+encrypted_b</code>	<code>EncryptedNumber</code> 对象重载后的加法运算，对应 Paillier 的加法同态，可得到 $E(a+b)$ 。
<code>encrypted_a*k</code>	<code>EncryptedNumber</code> 与明文整数的数乘，对应 Paillier 的标量乘法，可得到 $E(k \cdot a)$ 。
<code>encrypted_number.ciphertext(be_secure=True)</code>	返回密文对应的大整数表示，便于调试、显示以及序列化传输。

phe 已经把 Paillier 的核心操作封装得比较完整，直接使用 Python 运算符就能够实现同态加法和数乘，这为后续编写 PIR 模拟程序提供了很大便利。

3.3 隐私信息获取协议模拟

由于本实验主要是为了学习 Paillier 加密算法，所以没有引入真实的网络通信，而是在同一个 Python 程序中分别模拟客户端和服务器的行为。服务器端先保存若干条整数消息，客户端选择目标位置后构造加密的 one-hot 选择向量，即目标位置对应 $E(1)$ ，其他位置对应 $E(0)$ 。服务器根据收到的密文向量进行同态计算，最后返回结果密文，客户端再用私钥解密得到目标消息。

其整体流程可以概括为以下几步：

1. 服务器初始化消息列表 `MESSAGE_LIST`；
2. 客户端调用 `generate_paillier_keypair()` 生成公私钥；
3. 客户端利用 `encrypt()` 构造加密选择向量；
4. 服务器通过 `encrypted_bit * message` 和密文加法计算响应；
5. 客户端调用 `decrypt()` 解密响应，并验证结果是否等于目标消息。

这里需要说明一点，程序在服务器端采用的计算形式为

$$R = \sum_{j=1}^m M_j \cdot E(s_j),$$

其效果与理论部分给出的

$$R = \prod_{j=1}^m c_j^{M_j}$$

两者本质上是等价的。原因是 phe 已经把 Paillier 的相关同态运算封装进了 Python 运算符中，因此程序中直接写成“密文乘明文后再相加”会更容易理解，也更方便调试。

(1) 参数设置与辅助函数

这里主要给出**实验参数**，并额外实现了一个密文预览函数。由于 Paillier 密文本质上是整数，直接完整输出会非常长，因此程序只截取前后若干位进行展示，便于观察实验现象。

Listing 1: 参数设置与密文预览辅助函数

```
1 from time import perf_counter
2
3 from phe import paillier
4
5
6 # 服务器保存的消息列表。基础版实验中直接使用整数消息，便于验证结果。
7 MESSAGE_LIST = [135, 246, 357, 468, 579, 680, 791, 802]
8 # 客户端想要获取的目标下标，采用从 0 开始的计数方式。
9 TARGET_INDEX = 4
10 # 为了减小随机波动，耗时测试默认连续运行 10 次并取平均值。
11 BENCHMARK_RUNS = 10
12
13
14 def preview_ciphertext(encrypted_number, head=18, tail=18):
15     """返回密文的大整数预览，避免终端输出过长。"""
16     value = str(encrypted_number.ciphertext())
17     if len(value) <= head + tail:
18         return value
19     return f"{value[:head]}...{value[-tail:]}"
```

(2) 客户端构造加密查询向量

下面代码对应**客户端操作**。程序根据目标下标生成 one-hot 选择向量，再逐项加密，得到可以发送给服务器的查询密文。

Listing 2: 客户端构造加密查询向量

```
1 def build_encrypted_query(public_key, target_index, message_count):
2     """构造加密后的 one-hot 选择向量。"""
3     selection_vector = []
4     encrypted_query = []
5
6     for index in range(message_count):
7         # 目标位置为 1，其余位置为 0；服务器最终只能看到其加密结果。
```

```
8     bit = 1 if index == target_index else 0
9     selection_vector.append(bit)
10    encrypted_query.append(public_key.encrypt(bit))
11
12    return selection_vector, encrypted_query
```

(3) 服务器在密文域中计算响应

这一部分对应**服务器操作**。服务器虽然不知道客户端查询的目标位置，但仍然可以根据同态性质对密文选择位和消息做运算，最终得到目标消息的密文结果。

Listing 3: 服务器执行同态聚合计算

```
1 def server_compute_response(messages, encrypted_query):
2     """服务器在密文域中计算响应，但无法得知客户端真正查询的位置。"""
3     response = None
4
5     for message, encrypted_bit in zip(messages, encrypted_query):
6         # encrypted_bit * message 对应  $E(s_j)$  的标量乘法，得到  $E(s_j * M_j)$ 。
7         term = encrypted_bit * message
8         # 再利用加法同态把所有项相加，最终得到  $E(\sum(s_j * M_j))$ 。
9         response = term if response is None else response + term
10
11    return response
```

(4) 完整 PIR 流程封装

这里把生成密钥、构造查询、服务器计算和客户端解密这些步骤串起来，形成一次**完整实验**。

Listing 4: 同态验证与完整 PIR 流程封装

```
1 def run_basic_pir_demo(messages, target_index):
2     """串联完整的基础版 PIR 流程，并记录各阶段耗时。"""
3     timings = {}
4     # 第一步：客户端生成 Paillier 公私钥。
5     start = perf_counter()
6     public_key, private_key = paillier.generate_paillier_keypair()
7     timings["keygen"] = perf_counter() - start
8     # 第二步：客户端构造加密选择向量并发送给服务器。
9     start = perf_counter()
10    selection_vector, encrypted_query = build_encrypted_query(
```

```
11     public_key, target_index, len(messages)
12 )
13 timings["query"] = perf_counter() - start
14 # 第三步：服务器在密文上完成聚合计算，返回目标消息对应的密文。
15 start = perf_counter()
16 response = server_compute_response(messages, encrypted_query)
17 timings["server"] = perf_counter() - start
18 # 第四步：客户端用私钥解密服务器响应，恢复目标消息。
19 start = perf_counter()
20 decrypted_message = private_key.decrypt(response)
21 timings["decrypt"] = perf_counter() - start
22 return {
23     "public_key": public_key,
24     "selection_vector": selection_vector,
25     "encrypted_query": encrypted_query,
26     "response": response,
27     "decrypted_message": decrypted_message,
28     "expected_message": messages[target_index],
29     "timings": timings,
30 }
```

(5) 平均耗时统计

这里探究本算法的时间花销，考虑到 Paillier 密钥生成具有一定随机性，单次运行时间可能会有波动，因此程序额外实现了重复测试函数，对各阶段耗时取平均值，便于后续分析性能。

Listing 5: 连续运行 10 次并统计平均耗时

```
1 def benchmark_average_timings(messages, target_index, runs):
2     """连续运行多次实验，对各阶段耗时取平均值。"""
3     timing_keys = ["keygen", "query", "server", "decrypt"]
4     totals = {key: 0.0 for key in timing_keys}
5
6     for _ in range(runs):
7         result = run_basic_pir_demo(messages, target_index)
8         for key in timing_keys:
9             totals[key] += result["timings"][key]
10
11     return {key: totals[key] / runs for key in timing_keys}
```

(6) 主程序输出实验结果

最后，主函数统一组织实验输出，会先给出一次完整运行的结果，再给出多次重复实验后的平均耗时，用于同时观察功能正确性和运行开销。下面是实验输出结果。

```
1 ===== 基于 Paillier 的基础版 PIR 模拟 =====
2 消息列表: [135, 246, 357, 468, 579, 680, 791, 802]
3 目标位置(从 0 开始): 4
4 目标消息: 579
5 1. 客户端生成密钥
6   公钥模数 n 的十进制位数: 925
7   耗时: 11.939428 s
8 2. 客户端构造加密选择向量
9   明文选择向量: [0, 0, 0, 0, 1, 0, 0, 0]
10  前三个查询密文预览: ['158196131542726835...632904040640303105',
11                        '242707681406516261...738274100717428844',
12                        '179010463652378832...287309945275481691']
13 耗时: 3.122870 s
14 3. 服务器在密文上执行同态计算
15  返回密文预览: 440153783050162662...595363797849386538
16  耗时: 0.011254 s
17 4. 客户端解密服务器响应
18  解密结果: 579
19  是否与目标消息一致: True
20  耗时: 0.107023 s
21 5. 连续运行 10 次后的平均耗时
22  平均密钥生成耗时: 4.284174 s
23  平均查询向量构造耗时: 3.199725 s
   平均服务器同态计算耗时: 0.011667 s
   平均客户端解密耗时: 0.110424 s
```

(7) 实验结果分析

在模拟实验中，服务器消息列表设置为

[135, 246, 357, 468, 579, 680, 791, 802],

客户端查询的位置为第 4 号下标（从 0 开始计数），因此目标消息应为 579。程序的一次运行结果如下：

```
1 目标消息：579
2 明文选择向量：[0, 0, 0, 0, 1, 0, 0, 0]
3 解密结果：579
4 是否与目标消息一致：True
5 平均密钥生成耗时：4.284174 s
6 平均查询向量构造耗时：3.199725 s
7 平均服务器同态计算耗时：0.011667 s
8 平均客户端解密耗时：0.110424 s
```

从结果可以看出，基础版程序能够正确完成一次完整的 PIR 查询。客户端构造出的明文选择向量中，只有目标位置对应 1，其余位置均为 0，最终解密结果为 579，与目标消息完全一致，说明该实现已经满足“在不泄露查询位置的前提下正确获取目标消息”的基本要求。

从性能角度看，基础版程序的主要开销仍然集中在客户端。连续运行 10 次后的平均结果表明，密钥生成耗时为 4.284174 s，查询向量构造耗时为 3.199725 s，这两个阶段明显高于服务器端同态计算和客户端解密阶段。相比之下，服务器端同态聚合平均只需 0.011667 s，客户端最终解密平均为 0.110424 s。由此可以看出，在当前实验规模下，影响程序效率的主要因素仍然是 Paillier 密钥生成和多次加密操作，而服务器端的计算压力相对较小。

表 2: 基础版 PIR 程序各阶段平均耗时（连续运行 10 次）

阶段	耗时 (s)
密钥生成	0.491818
查询向量加密构造	0.380635
服务器同态计算	0.001904
客户端解密	0.014280

3.4 扩展: 获取对称加密的密文模拟

扩展实验在基础版的基础上继续往前推进。此时服务器保存的已经不是明文整数，而是客户端先用对称密钥加密后的密文。由于 Paillier 适合处理小于模数 n 的整数，而实际 AES 密文可能较长，所以程序中还额外加入了分块和重组逻辑，主要思路如下：

1. 客户端先生成 256 位 AES 密钥，并用 AES-GCM 对每条消息加密，服务器仅保存 `nonce + ciphertext`；
2. 程序根据 Paillier 公钥模数大小，自动计算单块可编码的最大字节数；
3. 若某条 AES 密文长度超过单块上限，则将其拆分为多个整数块；
4. 服务器将所有消息按“块列”组织，每一列执行一次基础版 PIR 聚合；
5. 客户端解密各列响应后重组目标密文，再使用本地 AES 密钥恢复原始明文。

这种设计比较符合实际应用习惯，即公钥同态部分负责隐藏查询行为，对称密钥部分负责保护消息内容，两者分工更加明确。

(1) 进阶扩展部分

由于查询构造、服务器聚合和基础版是一致的，这里不再重复展示。扩展版相对基础版新增的内容主要集中在“对称加密”“密文分块”和“结果重组”三个部分，因此下面分开说明。

(a) 对称加密消息

这一部分先用同一把 AES 密钥对所有消息进行加密。程序会为每条消息单独生成随机 `nonce`，然后把 `nonce + ciphertext` 一起保存下来，作为后续服务器端存储的密文内容。

Listing 6: 扩展版中的 AES 消息加密

```
1 def encrypt_messages_with_aes(messages, key):
2     """客户端用同一对称密钥加密消息，返回 nonce+ciphertext."""
3     aesgcm = AESGCM(key)
4     encrypted_messages = []
5
6     for message in messages:
7         nonce = token_bytes(AES_NONCE_SIZE)
8         ciphertext = aesgcm.encrypt(nonce, message.encode("utf-8"), None)
9         encrypted_messages.append(nonce + ciphertext)
10
11     return encrypted_messages
```

(b) 密文分块与服务器端组织

由于 AES 密文长度可能超过 Paillier 明文允许的范围，因此程序先根据公钥模数自动计算单块大小，再把字节串切分为若干整数块。之后，服务器端会把所有消息的分块结果按列组织起来，便于后续逐块执行 PIR 查询。

Listing 7: 扩展版中的密文分块与服务器端块组织

```
1 def compute_chunk_size(public_key):
2     """根据 Paillier 模数计算安全的单块字节数，确保分块整数小于 n."""
3     modulus_bytes = (public_key.n.bit_length() - 1) // 8
4     return max(1, modulus_bytes - 1)
5
6
7 def chunk_bytes(data, chunk_size):
8     """把字节串按固定长度切分为整数块."""
9     blocks = []
```

```
10
11     for start in range(0, len(data), chunk_size):
12         chunk = data[start : start + chunk_size]
13         blocks.append(int.from_bytes(chunk, "big"))
14
15     return blocks
16
17
18 def prepare_server_chunks(encrypted_messages, chunk_size):
19     """将所有 AES 密文补齐后按列组织，便于多轮 PIR 取回。"""
20     chunked_messages = [chunk_bytes(item, chunk_size) for item in
21                          encrypted_messages]
22     max_chunks = max(len(item) for item in chunked_messages)
23     chunk_matrix = []
24     length_list = [len(item) for item in encrypted_messages]
25
26     for blocks in chunked_messages:
27         row = blocks + [0] * (max_chunks - len(blocks))
28         chunk_matrix.append(row)
29
30     columns = []
31     for column_index in range(max_chunks):
32         columns.append([row[column_index] for row in chunk_matrix])
33
34     return {
35         "chunk_matrix": chunk_matrix,
36         "columns": columns,
37         "length_list": length_list,
38         "max_chunks": max_chunks,
39     }
```

(c) 密文重组

客户端从服务器取回并解密的是若干个整数块，因此还需要把这些块重新恢复成原始字节串。这程序根据块的位置判断应恢复的字节长度，尤其是最后一个块需要按实际长度截取，避免重组结果出错。

Listing 8: 扩展版中的密文重组

```
1 def reconstruct_bytes(blocks, original_length, chunk_size):
2     """将解密出的整数块恢复为原始字节串。"""
3     recovered = bytearray()
4     block_count = len(blocks)
5
6     for index, block in enumerate(blocks):
7         if index == block_count - 1:
8             block_length = original_length - chunk_size * (block_count - 1)
9         else:
10            block_length = chunk_size
11            block_bytes = block.to_bytes(block_length, "big")
12            recovered.extend(block_bytes)
13
14     return bytes(recovered)
```

(2) 进阶实验结果分析

在进阶实验中，服务器不再直接保存明文消息，而是保存客户端事先利用 AES-256-GCM 加密后的密文。客户端查询的目标位置设为下标 2。为了验证分块恢复流程，程序将目标消息设置为较长文本，使其对应的 AES 密文在当前参数下会被拆成 2 个整数块。程序的一次运行结果摘要如下：

```
1 消息总数：4
2 目标位置(从 0 开始)：2
3 目标密文共分块：2
4 恢复出的目标密文是否一致：True
5 是否与目标消息一致：True
6 平均密钥生成耗时：1.913527 s
7 平均 AES 预加密耗时：0.000396 s
8 平均查询向量构造耗时：0.303442 s
9 平均服务器同态计算耗时：0.177492 s
10 平均 Paillier 解密耗时：0.053601 s
11 平均 AES 解密耗时：0.000117 s
```

从结果可以看出，进阶实验中的整体流程同样能够正确完成。客户端先通过多轮 PIR 恢复目标 AES 密文，再利用本地保存的对称密钥将其解密为原始明文；恢复出的目标密文与原始目标密文一致，最终明文也与目标消息一致，这说明“服务器保存对称密文，客户端通过 PIR 取回后再解密”的方案是可行的。

与基础版相比，进阶实验最明显的变化是加入了分块处理。由于目标密文被拆分成了 2 个整数块，

服务器需要针对每个块分别执行一次 PIR 聚合，因此服务器端同态计算和客户端的 Paillier 解密开销都有所增加。平均结果表明，Paillier 密钥生成耗时为 1.913527 s，查询向量构造耗时为 0.303442 s，服务器同态计算耗时为 0.177492 s，Paillier 解密耗时为 0.053601 s。相比之下，AES 本身的加解密开销极小，AES 预加密和 AES 解密平均仅为 0.000396 s 和 0.000117 s。

因此，进阶实验反映出一个比较清楚的结论：在这种“Paillier + 对称加密”的组合方案中，真正影响整体性能的仍然是 Paillier 相关操作，而不是 AES。本实验已经完成了题目要求中的扩展部分，同时也说明如果消息长度进一步增加，分块数量还会继续增多，系统总开销也会随之上升。

表 3: 扩展版 PIR 程序各阶段平均耗时（连续运行 5 次）

阶段	耗时 (s)
Paillier 密钥生成	1.913527
AES 预加密	0.000396
查询向量加密构造	0.303442
服务器同态计算	0.177492
Paillier 解密恢复密文	0.053601
AES 解密恢复明文	0.000117

4 实验总结 & 心得体会

通过这次实验，我对 Paillier 算法的理解比只看公式时更直观了。尤其是在实际编程过程中，可以清楚看到“客户端构造加密选择向量”“服务器在密文域上完成运算”“客户端再解密取回结果”这一整套流程是如何对应理论内容的。另外，扩展实验也让我认识到，在真实应用里同态加密和对称加密并不是互相替代的关系，而是可以结合起来使用：前者负责隐藏查询行为，后者负责高效保护数据内容。整体来看，这次实验不仅帮助我理解了同态加密的基本原理，也让我对隐私查询协议的实现方式有了更具体的认识。

参考文献

- [1] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'99, page 223–238, Berlin, Heidelberg, 1999. Springer-Verlag.
- [2] Ronald L. Rivest, Len Adleman, and Michael L. Dertouzos. On data banks and privacy homomorphisms. In Richard A. DeMillo and Richard J. Lipton, editors, *Foundations of Secure Computation*, pages 169–179. Academic Press, New York, 1978.